list of methods is usually quite long. You can use Array#grep to check that my_method is in obj's list:

```
obj.methods.grep(/my/)  # => [:my_method]
```

If you could pry open the Ruby interpreter and look into obj, you'd notice that this object doesn't really carry a list of methods. An object contains its instance variables and a reference to a class (because every object belongs to a class, or—in OO speak—is an *instance* of a class)…but no methods. Where are the methods?

Your pair-programming buddy Bill walks over to the nearest whiteboard and starts scribbling all over it. "Think about it for a minute," he says, drawing the following diagram. "Objects that share the same class also share the same methods, so the methods must be stored in the class, not the object."

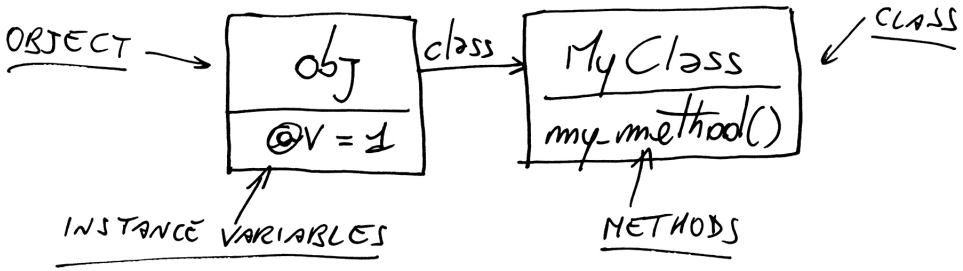


Figure 1—Instance variables live in objects; methods live in classes.

Before going on, you should be aware of one important distinction about methods. You can rightly say that "obj has a method called my_method," meaning that you're able to call obj.my_method(). By contrast, you shouldn't say that "MyClass has a method named my_method." That would be confusing, because it would imply that you're able to call MyClass.my_method() as if it were a class method.

To remove the ambiguity, you should say that my_method is an *instance method* (not just "a method") of MyClass, meaning that it's defined in MyClass, and you actually need an object (or *instance*) of MyClass to call it. It's the same method, but when you talk about the class, you call it an *instance method*, and when you talk about the object, you simply call it a *method*. Remember this distinction, and you won't get confused when writing introspective code like this:

```
String.instance_methods == "abc".methods    # => true
String.methods == "abc".methods             # => false
```